MASTER PROMPT — BUILD COMPLETE SAHAYYAK AI PROJECT

You are an expert full-stack software development team consisting of:

- Senior Product Architect
- UI/UX Designer
- Frontend Developer
- Backend Developer
- Database Engineer
- AI/LLM Engineer
- Accessibility Specialist
- Security Engineer
- QA/Test Engineer
- DevOps/Deployment Engineer

Your task is to **design, develop, test, debug, document, and prepare a complete working project called "Sahayyak AI" from scratch**.

Do NOT only give me code examples or instructions.

You must **create and modify the actual project files inside the current Antigravity workspace**.

Continue working through all development phases until a functional, polished MVP is available.

---

1. PROJECT NAME

Sahayyak AI

Suggested full title:

Sahayyak AI — An AI-Powered Digital Assistant for Elderly and Technology-Inexperienced Users

---

2. PROJECT PURPOSE

Sahayyak AI is a simple, accessible, AI-powered web platform designed for:

- Elderly people
- People with limited technical knowledge

- First-time smartphone/internet users
- People who are uncomfortable using modern digital services
- Users who prefer instructions in simple language
- Users who may prefer voice interaction instead of typing

The main goal is:

Make technology easier, safer, and more understandable for everyone.

The system should explain complicated digital tasks in **very simple, step-by-step language**.

Example:

Instead of saying:

"Navigate to Settings > Accounts > Security > Two-Factor Authentication."

Sahayyak AI should say:

"Step 1: Open your phone's Settings app.

Step 2: Look for the Security option.

Step 3: Tap Security.

Step 4: Look for Two-Step Verification."

The application must prioritize simplicity over technical complexity.

---

3. CORE PRODUCT VISION

The user should be able to open Sahayyak AI and immediately understand:

1. What the application does
2. How to ask for help
3. How to use voice input
4. How to select a language
5. How to find common technology guides

The application should feel:

- Simple
- Friendly
- Trustworthy
- Accessible
- Modern
- Safe

- Non-intimidating

Avoid unnecessary animations, complicated menus, excessive text, and technical terminology.

---

4. TARGET USERS

Design the system primarily for:

Primary Users

- Elderly users
- Technology beginners
- Users with low digital literacy

Secondary Users

- Family members helping elderly users
- Caregivers
- Teachers
- Community workers
- Digital literacy organizations

---

5. TECHNOLOGY STACK

Use the following stack unless there is a strong technical reason to change something.

Frontend

- React
- Vite
- Tailwind CSS
- React Router
- JavaScript or TypeScript
- Axios
- Lucide React icons

Backend

- Python
- FastAPI
- Pydantic
- SQLAlchemy

- Uvicorn

Database

Use:

- PostgreSQL

Design the application so SQLite can optionally be used for easy local development/testing.

AI

Use:

- Google Gemini API

Create a clean AI service abstraction so the AI provider can be changed later.

Authentication

Use:

- JWT authentication
- Secure password hashing
- Role-based access where required

Deployment

Frontend:

- Vercel

Backend:

- Render or another suitable cloud platform

Database:

- PostgreSQL-compatible cloud database

---

6. PROJECT ARCHITECTURE

Create a clean production-style structure.

Suggested structure:

sahayyak-ai/

frontend/

backend/

database/

docs/

tests/

.env.example

.gitignore

README.md

docker-compose.yml

Do not blindly copy this structure if a better architecture is appropriate.

Use separation of concerns.

---

7. FRONTEND REQUIREMENTS

Create a responsive React application.

The frontend should work properly on:

- Desktop
- Laptop
- Tablet
- Mobile

Use Tailwind CSS.

Create reusable components.

Suggested components:

- Navbar
- Footer
- Button
- Card
- Modal
- SearchBar

- LanguageSelector
- VoiceButton
- ChatMessage
- ChatInput
- StepGuide
- GuideCard
- CategoryCard
- Alert
- LoadingSpinner
- ErrorMessage
- ConfirmationDialog

---

8. MAIN PAGES

Create at minimum:

Home Page

Include:

- Sahayyak AI logo/name
- Simple welcome message
- Large "Ask Sahayyak AI" button
- Voice interaction button
- Search
- Popular help categories
- Common questions
- Language selector
- Accessibility controls

Example:

"Hello! How can I help you today?"

Buttons:

- Ask a Question
- Speak to Sahayyak
- Learn Technology
- Popular Guides

---

9. AI CHAT PAGE

Create a dedicated AI assistant interface.

The user should be able to:

- Type a question
- Use voice input
- Receive an AI response
- Listen to the response
- Ask follow-up questions
- Clear conversation
- Start a new conversation

Example user question:

"How do I send a photo on WhatsApp?"

AI should respond with simple steps:

1. Open WhatsApp.
2. Open the person's chat.
3. Tap the attachment/photo icon.
4. Select the photo.
5. Tap Send.

Avoid unnecessarily technical explanations.

---

10. AI RESPONSE DESIGN

AI responses should support structured answers.

Use:

- Short introduction
- Numbered steps
- Important warnings
- Tips
- Optional related guides

Example:

How to send a photo on WhatsApp

Step 1

Open WhatsApp.

Step 2

Open the chat of the person you want to send the photo to.

Step 3

Tap the photo or attachment button.

Step 4

Choose your photo.

Step 5

Tap Send.

Safety Tip

Only send personal photos to people you trust.

---

11. VOICE FEATURES

Implement voice interaction where browser capabilities allow it.

Use browser Web Speech APIs where appropriate.

Features:

Voice Input

User presses a microphone button and speaks.

Convert speech to text.

Send the text to the AI.

Voice Output

Add a speaker button to AI responses.

Read the answer aloud.

The user should be able to:

- Start speaking
- Stop speaking
- Replay response
- Pause speech if supported

Handle unsupported browsers gracefully.

Do NOT make voice functionality mandatory for the application to work.

---

12. MULTILINGUAL SUPPORT

Implement multilingual architecture.

Initially support:

- English
- Hindi
- Marathi

Create translation files rather than hardcoding all UI text.

Suggested:

frontend/src/locales/

en.json

hi.json

mr.json

The language selector should allow users to change languages.

AI responses should preferably be returned in the user's selected language.

Ensure Marathi and Hindi Unicode rendering works correctly.

---

13. TECHNOLOGY HELP CATEGORIES

Create categories such as:

Smartphone

- Change phone settings
- Connect Wi-Fi
- Turn Bluetooth on/off
- Install an app
- Update an app
- Take a screenshot

- Change font size
- Set an alarm

WhatsApp

- Send a message
- Send a photo
- Send a video
- Make a voice call
- Make a video call
- Create a group
- Block a contact
- Delete a message

Email

- Create email
- Send email
- Reply to email
- Attach a file
- Download attachment
- Recognize suspicious emails

Online Payments

Provide educational guidance for:

- Understanding UPI
- Sending money
- Receiving money
- Checking transaction status
- Understanding payment notifications

IMPORTANT:

Never request or store:

- UPI PIN
- ATM PIN
- CVV
- OTP
- Passwords
- Bank credentials

Clearly warn users not to share these.

Government/Digital Services

Provide general guidance for:

- Searching official websites
- Creating accounts
- Understanding online forms
- Downloading documents

Do not pretend to be an official government service.

---

14. GUIDE LIBRARY

Create a searchable guide library.

Each guide should contain:

- Title
- Category
- Difficulty
- Description
- Steps
- Safety tips
- Related guides
- Language

Example:

Guide:

"How to Connect to Wi-Fi"

Difficulty:

"Easy"

Steps:

1. Open Settings.
2. Tap Wi-Fi.
3. Turn Wi-Fi on.
4. Select your network.
5. Enter the password.
6. Tap Connect.

---

15. SEARCH FUNCTIONALITY

Create a simple search system.

Users can search:

"WhatsApp"

"Wi-Fi"

"payment"

"email"

"photo"

"video call"

Search should return:

- Relevant guides
- Categories
- AI assistant option

Use debouncing where appropriate.

---

16. AI BACKEND

Create an AI service in FastAPI.

Suggested endpoint:

POST /api/ai/chat

Request:

- message
- language
- conversation history
- optional category

Response:

- answer
- language
- safety warning if applicable
- related guides if available

Create a dedicated AI service layer.

Do not place Gemini API logic directly inside route handlers.

---

17. GEMINI PROMPT ENGINEERING

Create a carefully designed system prompt.

The AI assistant should behave like a patient digital teacher.

Rules:

1. Use simple language.
2. Avoid technical jargon.
3. Explain one step at a time.
4. Use numbered instructions.
5. Ask a clarification question when necessary.
6. Do not overwhelm the user.
7. Never ask for sensitive credentials.
8. Never request OTPs.
9. Never request passwords.
10. Never request UPI PIN.
11. Never request CVV.
12. Warn users about scams when relevant.
13. Do not claim to be an official bank/government representative.
14. If the AI is uncertain, clearly say so.
15. Encourage users to use official applications and websites.
16. Keep responses concise unless the user asks for more detail.
17. For elderly users, prefer extremely clear instructions.
18. Explain unfamiliar terms.
19. Avoid making financial, medical, or legal decisions for users.
20. For dangerous situations, advise contacting the appropriate authority/professional.

---

18. SAFETY SYSTEM

Create a safety layer before sending requests to the AI where practical.

Detect potentially sensitive information such as:

- Passwords
- OTP
- PIN
- CVV
- Bank account credentials
- Card details

If a user attempts to provide such information, display a warning.

Example:

"Please don't share your OTP, PIN, password, or banking details with anyone, including Sahayyak AI."

Do not store sensitive information.

---

19. AUTHENTICATION

Implement:

- Register
- Login
- Logout
- JWT authentication
- Password hashing
- Protected routes
- User profile

User model should contain appropriate fields such as:

- id
- name
- email
- password_hash
- preferred_language
- created_at

Do not store plaintext passwords.

---

20. USER PROFILE

Create profile settings.

Users should be able to change:

- Name
- Preferred language
- Text size
- Voice preference
- Accessibility settings

Do not collect unnecessary personal information.

---

21. ACCESSIBILITY

Accessibility is one of the most important requirements.

Implement:

- Large buttons
- Large readable fonts
- High contrast
- Keyboard navigation
- Proper labels
- ARIA attributes
- Visible focus states
- Screen-reader-friendly elements
- Simple navigation
- Minimal clutter

Create a text-size control:

- Normal
- Large
- Extra Large

Make sure the interface remains usable when text is enlarged.

Follow WCAG accessibility principles as closely as practical.

---

22. EMERGENCY / SAFETY HELP

Create a safety/help section.

It should explain:

- Don't share OTP
- Don't share PIN
- Don't share passwords
- Beware of suspicious links
- Beware of fake customer-care numbers
- Don't install unknown remote-access applications
- Verify payment requests
- Use official websites/apps

Do NOT implement fake emergency calling or claim that Sahayyak AI can contact authorities unless that functionality actually exists.

---

23. DASHBOARD

After login, create a simple dashboard.

Include:

- Welcome message
- Ask AI
- Recent conversations
- Saved guides
- Popular guides
- Language
- Accessibility settings

Keep the dashboard extremely simple.

---

24. ADMIN FUNCTIONALITY

Create a basic admin role.

Admin can:

- Add guides
- Edit guides
- Delete guides
- View guide list
- Manage categories

Do not give ordinary users admin privileges.

Protect admin APIs.

---

25. DATABASE DESIGN

Create proper database models.

Suggested tables:

users

guides

categories

conversations

messages

saved_guides

Use foreign keys where appropriate.

Add:

- timestamps
- indexes where useful
- constraints
- validation

Create database migrations.

---

26. API DESIGN

Create clean REST APIs.

Suggested endpoints:

POST /api/auth/register

POST /api/auth/login

GET /api/users/me

PUT /api/users/me

POST /api/ai/chat

GET /api/guides

GET /api/guides/{id}

POST /api/guides

PUT /api/guides/{id}

DELETE /api/guides/{id}

GET /api/categories

POST /api/saved-guides

DELETE /api/saved-guides/{id}

GET /api/conversations

GET /api/conversations/{id}

DELETE /api/conversations/{id}

Use proper HTTP status codes.

---

27. ERROR HANDLING

Implement proper error handling.

Frontend should show friendly messages.

Never expose:

- Stack traces
- API keys
- Database credentials
- Internal server details

Example:

Instead of:

"AxiosError 500 Internal Server Error"

show:

"Sorry, something went wrong. Please try again."

---

28. ENVIRONMENT VARIABLES

Create:

.env.example

Include placeholders such as:

DATABASE_URL=

GEMINI_API_KEY=

JWT_SECRET=

CORS_ORIGINS=

Never hardcode API keys.

Never commit .env files containing secrets.

---

29. SECURITY

Implement reasonable security practices.

Include:

- Password hashing
- JWT validation
- Input validation
- CORS configuration
- Environment variables
- SQL injection protection through ORM
- Rate limiting where practical
- Request validation
- Secure error responses

Do not store sensitive credentials.

---

30. UI DESIGN

Create a professional but extremely simple UI.

Design principles:

- Large cards
- Large buttons
- Clear icons
- Simple navigation
- Rounded components where appropriate
- Good spacing
- High readability
- Minimal cognitive load

The design should feel like an application made specifically for people who are not comfortable with technology.

Do NOT make it look like a complicated developer dashboard.

---

31. HOME PAGE SECTIONS

Create:

1. Header
2. Hero section
3. Ask Sahayyak AI
4. Voice assistant
5. Popular categories
6. Popular guides
7. Safety tips
8. How Sahayyak AI works
9. Language support
10. Footer

---

32. RESPONSIVE DESIGN

Test:

- 320px mobile
- 375px mobile
- 425px mobile
- Tablet
- Laptop
- Desktop

Make sure:

- Buttons don't overflow
- Text remains readable
- Navigation works
- Chat interface works
- Cards adapt
- Voice controls remain accessible

---

33. LOADING STATES

Implement loading states for:

- AI responses
- Search
- Login
- Register
- Guide loading
- Saving guides

Use accessible loading indicators.

---

34. AI CHAT UX

Create a friendly chat interface.

User message:

"You are my assistant."

AI response:

"Of course! I'm here to help. Tell me what you want to do."

Add:

- Microphone button
- Send button
- Speaker button
- Clear chat
- New conversation
- Suggested questions

Suggested questions:

"How do I use WhatsApp?"

"How do I connect Wi-Fi?"

"How do I send money using UPI?"

"How do I create an email?"

---

35. CONVERSATION HISTORY

Store conversations for authenticated users.

Each conversation should contain:

- Title
- Created date
- Updated date
- Messages

Allow:

- Open previous conversation
- Delete conversation
- Start new conversation

Do not unnecessarily store sensitive information.

---

36. SAVED GUIDES

Allow users to bookmark guides.

Example:

"Save this guide"

Then show it under:

"My Saved Guides"

---

37. TESTING

Create tests.

Backend:

- Authentication tests
- API tests
- Database tests
- AI endpoint tests
- Validation tests

Frontend:

- Component tests

- Basic page tests
- Form validation tests

End-to-end:

- Register
- Login
- Ask AI
- Search guide
- Open guide
- Save guide
- Change language
- Change text size
- Logout

Fix all discovered errors.

---

38. API TESTING

Create an API testing collection if practical.

Document:

- Endpoint
- Method
- Authentication
- Request body
- Response
- Error cases

---

39. DOCUMENTATION

Create a professional README.md.

Include:

Project Overview

Features

Technology Stack

Architecture

Project Structure

Installation

Environment Variables

Database Setup

Running Backend

Running Frontend

Running Tests

Gemini API Setup

Deployment

Troubleshooting

Future Enhancements

---

40. PROJECT DOCUMENTATION

Inside docs/ create:

architecture.md

api-documentation.md

database.md

ai-system.md

security.md

testing.md

deployment.md

user-guide.md

---

41. DOCKER

Create Docker support if practical.

Create:

Dockerfile for frontend/backend as appropriate.

docker-compose.yml for local development.

The project should be easy to start.

---

42. DATABASE SEEDING

Create sample categories and guides.

Include at least:

10–20 useful technology guides.

Examples:

- How to connect Wi-Fi
- How to use WhatsApp
- How to make a WhatsApp call
- How to send a WhatsApp photo
- How to send an email
- How to attach a file
- How to install an application
- How to update an application
- How to use UPI safely
- How to identify suspicious messages
- How to take a screenshot
- How to increase text size
- How to turn Bluetooth on
- How to connect Bluetooth headphones
- How to use Google Maps
- How to search Google safely

Create sample data using a seed script.

---

43. SAMPLE USER EXPERIENCE

Scenario:

User opens Sahayyak AI.

Screen says:

"Hello! What would you like help with?"

User presses microphone.

User says:

"How can I send a photo on WhatsApp?"

System converts speech to text.

AI processes the question.

AI responds:

"Sure! I'll help you step by step."

Then:

1. Open WhatsApp.
2. Open the person's chat.
3. Tap the attachment/photo button.
4. Select the photo.
5. Tap Send.

The user presses the speaker icon.

The system reads the instructions aloud.

This entire flow should be smooth.

---

44. AI FALLBACK

If Gemini is unavailable:

Do not crash the application.

Display:

"I'm unable to connect to the AI right now. You can try one of these help guides."

Then show relevant guides.

---

45. OFFLINE-FRIENDLY GUIDES

The static guide library should remain usable even if the AI service is unavailable.

The application should not depend entirely on Gemini.

---

46. PERFORMANCE

Optimize:

- API requests
- Database queries
- React rendering
- Search
- AI calls

Avoid unnecessary API requests.

Use caching where appropriate.

Do not introduce unnecessary complexity.

---

47. CODE QUALITY

Follow:

- Clean code
- Meaningful names
- Modular architecture
- Reusable components
- Comments only where useful
- No unnecessary duplicated code

Do not create huge files containing everything.

---

48. LOGGING

Implement useful server-side logging.

Never log:

- Passwords
- OTPs
- PINs
- CVVs
- API keys
- Authentication tokens

---

49. SECURITY DISCLAIMER

Add an appropriate disclaimer:

"Sahayyak AI provides educational and general technology guidance. Always verify important information using official sources."

For financial-related questions:

"Never share your OTP, PIN, password, CVV, or banking credentials with anyone."

---

50. FUTURE-READY ARCHITECTURE

Design the project so future features can be added:

- Android application
- WhatsApp assistant
- More Indian languages
- Regional language voice support
- Image/screenshot understanding
- Family/caregiver accounts
- Personalized learning
- Tutorial videos
- Accessibility improvements
- More AI providers

Do not implement unnecessary future features now unless they are simple and useful.

---

51. DEVELOPMENT PROCESS

You MUST follow this development process.

PHASE 1 — REQUIREMENTS

Analyze the complete prompt.

Create:

docs/requirements.md

List:

- Functional requirements
- Non-functional requirements
- User roles
- Features
- Constraints
- Security requirements
- Accessibility requirements

PHASE 2 — ARCHITECTURE

Design:

- Frontend architecture
- Backend architecture
- Database architecture
- AI architecture
- Authentication architecture

Create:

docs/architecture.md

PHASE 3 — PROJECT SETUP

Create the complete directory structure.

Initialize:

- React/Vite frontend

- FastAPI backend
- PostgreSQL integration
- Testing setup
- Environment configuration

PHASE 4 — DATABASE

Implement:

- Models
- Relationships
- Migrations
- Seed data

Verify database functionality.

PHASE 5 — BACKEND

Implement:

- Authentication
- User APIs
- Guide APIs
- Category APIs
- Conversation APIs
- Saved-guide APIs
- AI API

Test APIs.

PHASE 6 — FRONTEND

Implement:

- Home
- Login
- Register
- Dashboard
- AI Chat
- Guide Library
- Guide Details
- Saved Guides

- Profile
- Admin pages

PHASE 7 — AI

Integrate Gemini.

Implement:

- System prompt
- User language
- Conversation history
- Safety filtering
- Error handling
- Fallback

PHASE 8 — VOICE

Implement:

- Speech-to-text
- Text-to-speech
- Browser compatibility handling

PHASE 9 — ACCESSIBILITY

Test:

- Font sizes
- Keyboard navigation
- Screen-reader labels
- Contrast
- Focus states
- Responsive behavior

PHASE 10 — TESTING

Run:

- Unit tests

- API tests
- Frontend tests
- Integration tests
- Build tests

Fix errors.

PHASE 11 — SECURITY REVIEW

Check:

- Authentication
- Authorization
- API keys
- Environment variables
- Sensitive information
- Input validation
- CORS
- SQL injection risks
- XSS risks

Fix issues.

PHASE 12 — UI/UX REVIEW

Inspect the application visually.

Ask:

"Could a 70-year-old person with very limited technology knowledge use this without assistance?"

If the answer is no, simplify the interface.

PHASE 13 — FINAL BUILD

Run production builds.

Verify:

Frontend build succeeds.

Backend starts successfully.

Database connects successfully.

Tests pass.

No obvious console errors.

---

52. IMPORTANT AUTONOMY RULE

You are authorized to:

- Create files
- Modify files
- Create folders
- Install dependencies
- Run commands
- Run tests
- Debug errors
- Refactor code
- Improve UI
- Fix configuration problems

Do NOT stop after creating a few files.

Do NOT simply tell me what commands I should run if you can execute them yourself.

If you encounter an error:

1. Read the error.
2. Identify the root cause.
3. Fix it.
4. Run the relevant command again.
5. Continue development.

Do not leave known errors unresolved.

---

53. DO NOT MAKE THESE MISTAKES

Do NOT:

- Hardcode API keys
- Store plaintext passwords
- Create fake API responses in the final implementation

- Leave major features as TODOs
- Create placeholder pages instead of functionality
- Use unnecessary libraries
- Build a complicated UI
- Ignore mobile responsiveness
- Ignore accessibility
- Ignore error handling
- Ignore security
- Ask me to manually create every file

If a third-party API requires credentials, create .env.example and clearly document where the key is required.

---

54. WHEN A DECISION IS REQUIRED

If a minor implementation decision is needed, choose the most sensible production-ready option yourself.

Do not repeatedly stop and ask me for small decisions.

Only ask me if a decision genuinely requires information that cannot reasonably be determined from this specification.

---

55. FINAL PROJECT CHECKLIST

Before considering the project complete, verify all of the following:

[ ] Frontend created

[ ] Backend created

[ ] Database configured

[ ] Authentication implemented

[ ] AI integration implemented

[ ] Voice input implemented

[ ] Voice output implemented

[ ] English supported

[ ] Hindi supported

[ ] Marathi supported

[ ] Guide library implemented

[ ] Search implemented

[ ] Saved guides implemented

[ ] Conversation history implemented

[ ] User profile implemented

[ ] Admin functionality implemented

[ ] Accessibility implemented

[ ] Safety warnings implemented

[ ] Responsive design implemented

[ ] Error handling implemented

[ ] Loading states implemented

[ ] Environment variables configured

[ ] Security reviewed

[ ] Tests created

[ ] Tests executed

[ ] Build tested

[ ] README created

[ ] API documentation created

[ ] Architecture documentation created

[ ] Database documentation created

[ ] AI documentation created

[ ] Deployment documentation created

[ ] Sample data created

[ ] Docker configuration created where appropriate

[ ] No known critical errors remain

---

56. FINAL DELIVERABLE

At the end, provide a concise development report containing:

1. Project overview
2. Features implemented
3. Technology stack
4. Project structure
5. Database structure
6. API endpoints
7. AI functionality
8. Voice functionality
9. Accessibility functionality
10. Security measures
11. Tests performed
12. Known limitations
13. Local setup instructions
14. Deployment instructions
15. Future improvements

Most importantly:

The actual working project must exist inside the Antigravity workspace.

Do not consider documentation alone a successful completion.

The success condition is:

A user should be able to clone/open the project, configure the required environment variables, start the frontend and backend, open Sahayyak AI in a browser, register/login, ask questions using text or voice, receive simple AI guidance, browse technology guides, save guides, change language, adjust accessibility settings, and use the application comfortably on mobile and desktop.**

BEGIN NOW.

First inspect the current workspace.

Then create the project architecture.

Then implement the project phase by phase.

Do not stop at planning.

Build the actual application.